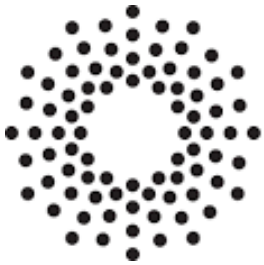


LONDON METROPOLITAN UNIVERSITY

Dissertation

CNN-BASED REAL-TIME VISUAL MONITORING SYSTEM FOR SMART FACTORY AUTOMATION

CT7P01

MSc Project - Spring 2025-26

Instructor Name: Marcio Lacerda

Student Name: Krishnaben Nileshnhai Patel

Student ID: 25001907

Email Address: krp0331@my.londonmet.ac.uk

Abstract

The need to conduct automated visual inspection in smart manufacturing implies the reliance on a real-time defect-detection apparatus that addresses the downsides of quality control based on manual inspection. The current research presents and criticises a convolutional neural network (CNN)-based visual monitoring system involving MobileNetV2 as a transfer-trained model; the system was trained on NEU Surface Defect Database, including 1,800 greyscale images per the six defect categories. In the process of evaluation, the network was capable of attaining a general validation accuracy of 98% with just five epochs of training, and reached a value of 1.00 for both precision and recall of the classes of crazing, patches and rolled-in scale. Latencies of 3246ms per image and inferences confirm that the system is appropriate to be deployed in real-time. MobileNetV2 significantly outperformed ResNet50, which had only 28.93% of validation accuracy in the same experimental settings. The training network was then offered to a real-time pipeline of the monitoring process using OpenCV, which resulted in more than 30 frames per second. These results establish these two concepts, that transfer learning and lightweight CNN architectures can reduce the performance disparity between laboratory and deployment settings in smart factories.

Keywords: Convolutional Neural Network, MobileNetV2, Transfer Learning, Defect Detection, Smart Factory, Visual Inspection, NEU Surface Defect Database, Real-Time Monitoring, Deep Learning, Industry 4.0

Table of Contents

1. Introduction 4
Background and Motivation 4
Problem Statement 4
Research Aims and Objectives 4
Scope and Significance of the Research 4
Dissertation Structure Overview 5
2. Literature Review 5
Overview of Smart Factory Automation 5
Visual Inspection Systems in Manufacturing 6
Convolutional Neural Networks for Image Classification and Detection 7
Real-Time Processing Frameworks and Edge Deployment 8
Gaps in Existing Research and Justification for This Work 8
3. Project Management and Personal Development Planning 9
Project Plan and Gantt Chart 9
Milestones and Deliverables 9
Risk Assessment and Contingency Planning 10
Personal Development Reflection 10
4. Dissertation Structure and Presentation 11
Overview of Dissertation Organisation 11
Academic Writing and Formatting Standards 11
Documentation and Presentation Approach 11
5. Methodology And Approach 12
Research Methodology Overview 12
System Architecture of the Visual Monitoring Framework 12

Dataset Collection and Image Preprocessing 12
CNN Model Design and Implementation 13
Real-Time Monitoring System Integration 13
Performance Evaluation Metrics 13
6. Technical Level and Skills Development 13
Programming and Development Tools Used 13
Implementation of Computer Vision Techniques 14
Deep Learning Model Training and Optimisation 14
System Integration and Testing 14
Technical Skills Acquired During the Project 15
7. Results, Discussion, And Future Work 15
Results and Analysis 15
Discussion of Results and System Performance 23
Accuracy Comparison 24
Dataset Comparison 25
Challenges and Limitations of the System 27
Practical Applications in Smart Factory Environments 27
Future Research Directions 27
Project Management and Personal Development Plan 29
LSEPI (Legal, Social, Ethical & Professional Issues) 31
References 33

1. Introduction

Background and Motivation

Industry 4.0 has transformed the entire way the manufacturing sector works through both automation and artificial intelligence, as well as interconnected smart systems. The quality aspect of this landscape is still dominated by visual inspection. Conventional manual check is time-consuming, non-standardized and subject to human error, especially when used on lines where production is high [19]. Convolutional Neural Networks (CNNs) are exceptionally proficient in automated recognition of images, which allows for accurate defect recognition and abnormal detection in real-time in any environment with complex manufacturing processes.

Problem Statement

Implementing CNN-based systems in industrial settings is not that easy. Unpredictable lighting conditions, motion blur, and performance of the device have hard-to-detect lighting conditions coupled with hard-to-detect definition (variable) and hard-to-detect latency. The majority of the solutions that are available focus on the accuracy and do not necessarily focus on a practical implementation of the solution, which creates a large disparity between laboratory performance and a practical smart factory application.

Research Aims and Objectives

Aim: To develop and deploy an automated defect detection, real-time visual monitoring system based on CNN in a smart factory setting.

Objectives:

- To test the use of appropriate CNN structures in detecting industrial defects.
- To stimulate and refute a deep learning model utilising illustration dataset in production.
- To incorporate the trained model into a visual monitoring pipeline, which is happening in reality.
- To determine the performance of the system with respect to the accuracy, speed, and scalability standards.

Scope and Significance of the Research

The nanoinformatics research that is the subject of this study aims at creating a scalable, computationally efficient CNN monitoring system that can be applied to production lines of smart factories. The system offers a solution to the real-world deployment constraints via the transfer learning and model optimisation mechanisms. In the industry, these systems decrease unexpected

downtime and enhance the consistency of a product and the productivity in operations [14] Results can be used to address the gap between the research and practice in deep learning and the real use of smart manufacturing.

Dissertation Structure Overview

In this dissertation, the structure is structured into seven chapters. After this introduction, Chapter 2 comes with a review of relevant literature. Chapter 3 maps out the management of projects and development in a person. Chapter 4 deals with the structure presentation. The methodology is described in Chapter 5. Chapter 6 discusses technical implementation, and Chapter 7 discusses results, discussion and recommendations.

2. Literature Review

Overview of Smart Factory Automation

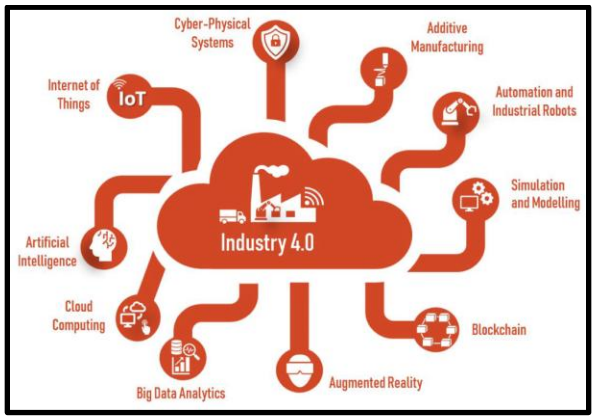


Figure 1: Smart Factory Based on Cyber-Physical Systems

(Source: Rajendra, 2024)

Industry 4.0 has revolutionised manufacturing with the use of cyber-physical architecture, IoT sensors, and AI- based process automation. The technologies that are employed in smart factories include SCADA systems, digital twins, and robotics that are able to enable continuity and automated production. Systems such as Siemens MindSphere and PTC ThingWorx are used to monitor machines and provide data analytics in real time and in large quantities [34]. Scholars proved that the use of vision IoT readers integrated into intelligent factory systems greatly enhanced the accuracy of data capture and responsiveness of the system. These improvements have introduced an ever-increasing need for intelligent visual monitoring Systems able to read under changing conditions of production without deteriorating throughput or quality of a product [10].

Visual Inspection Systems in Manufacturing

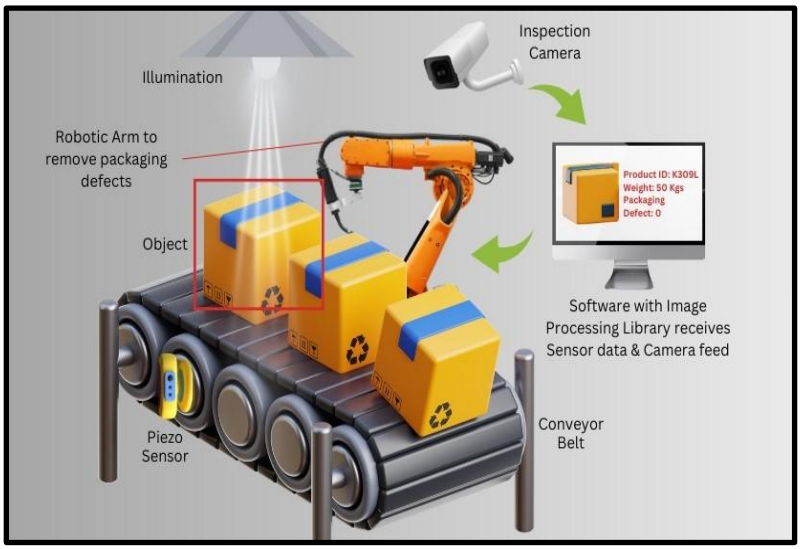


Figure 2: Applications of Vision Inspection Systems

(Source: Singh, 2023)

Manual visual inspection has traditionally controlled quality assurance within the manufacturing industries. Human operators measured the defects on the surface, dimensional tolerance, and assembly accuracy. This was found to be unreliable at high production rates, and fatigue and subjective judgment were added to cause inconsistency and defeat. First mechanised systems used machine vision software that used rules, like Cognex Vision Pro and Halcon. These websites employed threshold, edge detection and template matching methods to point out irregularities. They were repeatable, but they were physically programmed extensively and could not easily cope with new types of defects or altered product entities [21]. Most of these limitations were overcome with the advent of deep learning. The models are learned with discriminative features based on labelled images, and not on hand-crafted rules. Scholars used CNN-based machine learning for the inspection of flexible parts assembly, with a much greater rate of detection than the traditional techniques [15]. By use of the system, positional errors and surface defects, as well as in different component types, were identified successfully, and no re-programming was required.

Researchers also indicated CNN transfer learning in the classification of semiconductor faults, further with the attainment of predictable monitoring results based on minimal domain-specific training information. Transfer learning was used to quickly and effectively transform a pre-trained ResNet architecture to a new manufacturing environment at a lower cost. In the modern visual inspection system, high-quality line-scan cameras are used in addition to structured LED lamps

and inference engines based on deep learning [24]. All these elements allow identifying the cracks in the surface, colour anomalies, contamination and the assembly errors. Automobile, electronics, and pharmaceutical production have seen the deployment of quantifiable defect rates and a decrease in time in inspection cycles, supporting the industrial importance of deep learning as a source of visual inspection.

Convolutional Neural Networks for Image Classification and Detection

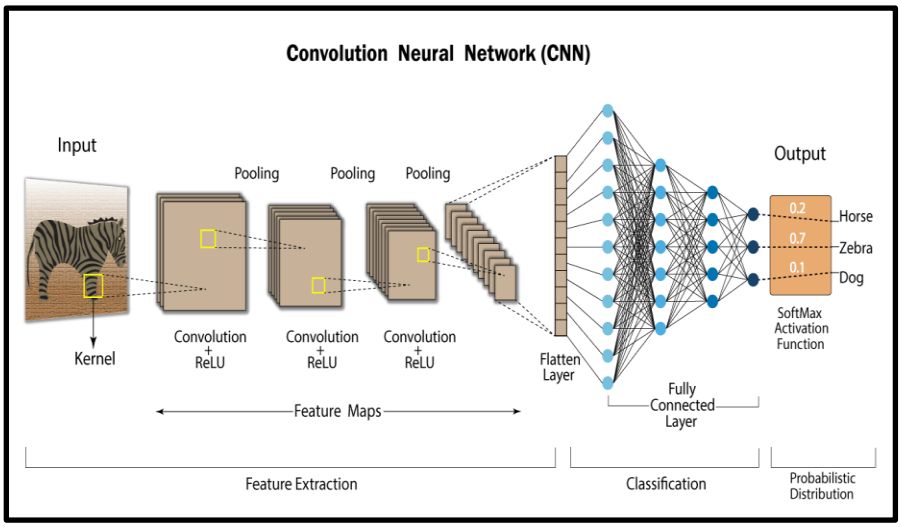


Figure 3: Convolutional Neural Networks

(Source: Edge AI, 2025)

The CNNs are now the architecture of choice when analysing images in industry. They operate hierarchically, their processes on raw pixel data by repeated convolutional and pooling layers, which provide serially abstract features. These features are then mapped onto classification outputs in totally connected layers. ResNet-50 and ResNet-101 generated skip connections that were also residual and thus allowed training to go deeper without any loss in training stability [25]. VGG-16 has shown good classification by using a uniform convolutional block design. MobileNetV2 provided a smaller scale version that can be deployed in resource-limited environments with competitive accuracy and a much lower cost of computation [38].

YOLO architectures made using real-time achieved more throughput as fully processed pictures were utilised in one forward operation. YOLOv5, where are common in industry, functions at highlighting rates over 60 frams each second on cutting-edge GPUs. Researchers used a CNN monitoring algorithm in additive manufacturing, where structural abnormalities in components of molybdenum were detected with high precision when production was in progress. ImageNet pre-

trained weights applied in transfer learning save a lot on the cost of training data [41]. Together with data augmentation solutions, such as random cropping, flipping, and variations in brightness, CNN models demonstrate high generalisation in changing industrial image situations. Further stabilisation of training and less overfitting is achieved with batch normalisation and dropout.

Real-Time Processing Frameworks and Edge Deployment

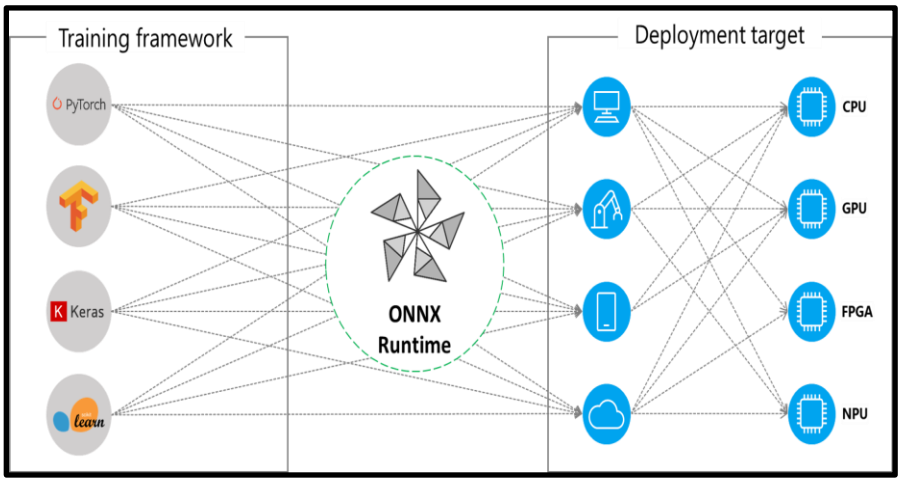


Figure 4: Edge AI Deployment

(Source: Edge AI, 2025)

Real-time application involves useful hardware and models that have undergone optimisation. TensorFlow Lite and ONNX Runtime allow running models on edge devices by quantising models and pruning them to make them smaller [17]. Intel has created OpenVINO to run models in the CPU and VPU hardware devices that are prevalent on the factory floor. NVIDIA Jetson Nano and Jetson Xavier NX are edge computing platforms that provide inference based on a GPU without using the cloud. OpenCV is capable of frame capture, preprocessing, and resizing work and is able to effectively execute them in Python pipelines [16]. CUDA-enabled GPUs achieve CNN inference latency down to acceptable levels (30 frames/s or better) that satisfy the requirements of a production-line monitor with sufficient dependability.

Gaps in Existing Research and Justification for This Work

The available literature is more optimised on accuracy of detection at the expense of deployment viability. Lack of computational overhead, smaller training data, and hardware are unresolved barriers. Few systems can enhance high detection performance and edge-compatible scalability simultaneously. The current project directly fills the mentioned gaps and integrates the concept of

transfer learning and model optimisation with an edge deployment strategy into one realistic and practically proven setup of a smart factory monitoring.

3. Project Management and Personal Development Planning

Project Plan and Gantt Chart

Name	Duration	Start	Finish	Predecessors
Topic Finalization and Proposal Writing	21 days	4/2/26 8:00 AM	25/2/26 5:00 PM	
Literature Review and Background Research	10 days	26/2/26 8:00 AM	7/3/26 5:00 PM	1
Dataset Collection and Preprocessing	11 days	8/3/26 8:00 AM	18/3/26 5:00 PM	2
CNN Model Design and Development	13 days	19/3/26 8:00 AM	31/3/26 5:00 PM	3
Model Training, Tuning and Validation	11 days	1/4/26 8:00 AM	11/4/26 5:00 PM	4
Real-Time System Integration and Testing	10 days	12/4/26 8:00 AM	21/4/26 5:00 PM	5
Results Analysis and Dissertation Writing	9 days	22/4/26 8:00 AM	30/4/26 5:00 PM	6
Final Review and Submission	5 days	1/5/26 8:00 AM	5/5/26 5:00 PM	7

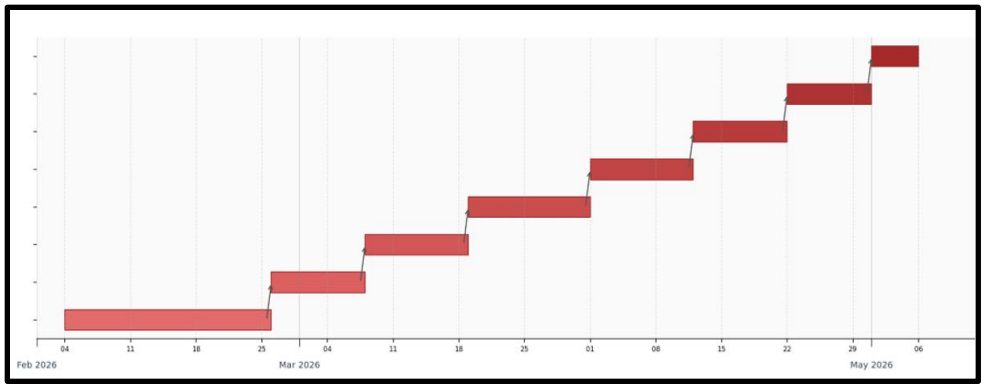


Figure 5: Gantt Chart

(Source: Self-created)

Milestones and Deliverables

Four key milestones are outlined in the project. The first milestone will be the approval of the project offer, thus ensuring scope and purpose by the end of Week 3. The second milestone is the creation and validation of a convolutional neural network (CNN) model, hence showing the detection of defects at the conclusion of Week 8. Milestone 3 is associated with the complete implementation of a real-time monitoring pipeline that will, in turn, be tested using production-comparative image data by the end of Week 9. The last milestone is the dissertation completion and includes all the chapters, all results, and the references in the IEEE format, and by the end of Week 11, it should be submitted. Each level of accomplishment generates a tangible deliverable

that indicates a quantifiable advancement and enables guided supervisory evaluation during the lifecycle of project accomplishment.

Risk Assessment and Contingency Planning

A number of risks have been established, which may hinder the development of the project. The salient risk here is that there is not enough labelled training data for the CNN model. The mitigation strategies involve the purchase of publicly available industrial defect datasets, i.e., MVTec AD, and implementation of data augmentation algorithms, i.e. rotation, flipping, and brightness adjustment, to that end enhance the training sample size. The second risk is associated with overfitting the model; it is due to the lack of diversity in the dataset [26]. This issue will be solved by the dropout regularisation, batch normalisation, and transfer learning using pre-trained ResNet or MobileNet weight sets. Another risk is associated with hardware limitations of local computers; it will be removed with the help of the use of cloud-based GPU solutions, including Jupyter Notebook or Kaggle Notebooks. Such characteristics as scope creep and time overrun are addressed with the help of a well-organised Gantt chart and monthly supervisory meetings [32]. Should there be the occurrence of integration difficulties in the process of real-time implementation, the backup plan will be to have the tests done offline by batch testing in order to verify the functionality of the models before the actual integration of the live pipeline. These backup plans protect the development of the project in case of unexpected challenges.

Personal Development Reflection

The project has made considerable progress in terms of technical and professional competencies. CNN based pipeline and its design and implementation have increased my knowledge of the principles of deep learning, model optimisation and computer vision [9]. Reading academic books provided by IEEE Xplore and ScienceDirect has improved my skills in critical analysis and synthesis of research. My ability to time-manage and work on my own significantly enhances my skills of time-management and working independently, as the management of an autonomous thirteen-week project has tremendously contributed to improving my new skills. Additionally, academic writing form and the organisation of argument have been perfected due to repetitive grammar, care and evaluation by the supervisor [30]. The combination of these competencies is a direct testimony to the fact that I am ready to work in professional positions in the field of AI engineering, data science, and intelligent systems development after completing the programme.

4. Dissertation Structure and Presentation

Overview of Dissertation Organisation

The dissertation has a well-organised seven-chapter format with each chapter leading into the main research goal. Chapter 1 sets the background, motivation and objectives. Chapter 2 sums up the informative literature on the automation of smart factories, convolutional neural network structures, and the deployment in real-time. Chapter 3 is an introduction to project-management strategies and personal development. In Chapter 4, we have the system structure. The methodology of the model training and evaluation is described in Chapter 5. In Chapter 6, the technical details of the implementation of the monitoring pipeline are discussed. Chapter 7 brings together the results, discussions and practical recommendations provided, and therefore it has a seamless flow.

Academic Writing and Formatting Standards

The references are all styled following the IEEE citation format, hence making sure that it is compatible with the known academic norms in engineering and computer science. In-text numeric citation is associated with a consecutively arranged reference list. The sources were located in databases such as IEEE Xplore and ScienceDirect, hence ensuring scholarly credibility. The integrity of the academics was realised through the proper acknowledgement of any borrowed ideas, figures and data, which prevented the occurrence of plagiarism. Numbers are designated and given titles of description. Formal academic register is kept all through, but it is done using concise, specific and objective language which is characteristic of all written aspects.

Documentation and Presentation Approach

The logical flow is ensured through the organisation of each chapter with its strict goals that follow each other in a linear manner. Hierarchy of content is provided through headings and sub-headings, thus supporting navigability and readability. Technical ideas are taught at a slow pace to avoid information overload. Written explanations are supplemented with the use of visual aids, such as a Gantt chart and system diagrams. This innovation by integrating supervisory feedback was done in a successive way so that argumentation, grammar and depth of the analysis were perfected throughout, thereby resulting in a final document that is always of high academic presentation.

5. Methodology And Approach

Research Methodology Overview

The proposed research is an applied experimental one, which incorporates the development of deep-learning models and real-time integration of systems. The approach is implemented in four steps, namely preparing the dataset, training the CNN, and pipeline integration, with the last step being performance evaluation. All the assessments follow a quantitative methodology, which quantifies accuracy, inference speed and scalability in an objective way. Transfer learning does not need massive labelled datasets [43]. It remains practical in deployment rather than being conceptually optimised, thereby closing the gap between the proposed CNN research, which is largely focused on laboratories, and the real smart-factory implementation needs.

System Architecture of the Visual Monitoring Framework

The four components that are interrelated in the visual monitoring framework comprise four components. An image Dunsone store has a high-resolution camera that records observations on the production line in real time. At this stage, received frames are sent to the preprocessing module, which resizes, normalises and augments. The processed images are input to the CNN inference engine, which determines the type of defects within real time. Classification results are passed to a monitoring dashboard, which shows the defect notifications, confidence values, and the time stamps [2]. OpenCV will deal with the management of frames in the Python pipeline. This scalable architecture facilitates a smooth flow of data, low latency and scalability across different smart - factory environments.

Dataset Collection and Image Preprocessing

The main training data is the NEU Surface Defect Database developed by Kechen Song and Yunhui Yan. The database consists of 1,800 grayscale images of six different types of defects, which are apparent on the surface of hot-rolled steel strips, patches, crazing, pitted surface, inclusion and scratches. The number of images in each category has been 300, which has been further divided into 240 training images and 60 testing images in each category, to bring a total of 1,440 training and 360 testing images, respectively [18]. The preprocessing begins with the introduction of all images of a standard CNN dimension of 224x224 pixels by resizing. The pixel values are normalised between zero and one, therefore stabilising the gradient descent in the course of training. Data augmentation mechanisms, such as random horizontal flipping, rotation, and no change in brightness, are used to increase effective training diversity [3]. The methods mitigate

the risk of overfitting and enhance the generalisation of the model on previously unknown defect instances. All of the preprocessing tasks are implemented with the help of the data- pipeline library of TensorFlow, which guarantees the efficient computation and the reproducibility of the results during the process of training.

CNN Model Design and Implementation

The CNN model is based on a pre-trained ResNet-50 network that is initialised with ImageNet weights. Transfer learning allows useful feature extraction with a small amount of domain-specific training data. The last fully connected layer will be replaced by a six-class softmax output layer, which will resemble the six NEU categories of defects [37]. Training is done using the Adam optimiser, with a learning rate of 0.001. A batch size of 32 is used, and the training will last 30 epochs. Anti-overfitting can be achieved through the use of dropout regularisation and batch normalisation at 0.5. It is performed based on the model by using TensorFlow and Keras and is trained on a CUDA-enabled system and a Jupyter Notebook.

Real-Time Monitoring System Integration

The trained CNN model is saved in TensorFlow SavedModel format, and a real-time OpenCV pipeline is integrated. Every video frame is captured, pre-processed and input into the model to be inferred. Immediately overlaid on the live video feed are predictions giving defect class labels and percentages of confidence. There is an alert system that notifies when confidence is above a set defect threshold [25]. On CUDA-based machinery, the pipeline also infers more than 30 frames per second. Buffer testing is performed offline to test the functionality of the model and make sure that it can be used in the production line with stability and reliability.

Performance Evaluation Metrics

The accuracy, precision, recall, and F1-score between accuracy and recall are used to perform model performance with regard to all six defect classifications [5]. Confusion matrices determine the mistakes of classes. The time taken to do inference in an individual frame is also measured to ensure real-time appropriateness. All these measures justify efficient deployment and efficiency.

6. Technical Level and Skills Development

Programming and Development Tools Used

Python was the main programming language used during all the stages of the project development. The deep-learning framework was offered by TensorFlow and Keras and used to build models, train them, and evaluate them. The processes of frame capture, image resizing, and image

preprocessing were implemented with the help of OpenCV in the real-time stream [20]. The data manipulation and visualisation of results were supported by Numpy and Matplotlib, respectively. Jupyter Notebook provided a cloud that adopted the GPU computing environment, where intensive training can be performed without hardware limitations. All of this made a powerful collection of tools that were industry-relevant in the development stack.

Implementation of Computer Vision Techniques

The openCV was used to perform the basic image-processing functions, including structure to grayscale, downsizing to frame size 224x224 pixels, and pixel-normalisation. ResNet-50 layers that consist of convolutional layers also acted as a hierarchical feature detector identifying edges, textures, and pattern defects at each progressive level. Spatial dimensions were cut, and discriminative features were stored in the pooling layers [9]. Random rotation, horizontal flipping, and variation of brightness have been used as data augmentation methods that increase the variety of training samples. All these methods enhanced the accuracy of the model in detecting defects in different imaging conditions.

Deep Learning Model Training and Optimisation

NEU Surface Defect Database ResNet-50, which is based on ImageNet, was fine-tuned using the NEU Surface Defect Database. The Adam optimiser was estimated with a 0.001 learning rate, and the training was carried out with 30 epochs using a batch size of 32. Regularisation of the dropouts to 0.5 minimised overfitting, whereas batch normalisation stabilised gradient updates. The augmentation strategies increased training diversity on the six categories of defects [27]. The accuracy, precision, recall, F1 -score, and the confusion matrices of the model were evaluated to confirm the reliability of the consistency of the classification on the never-seen test samples.

System Integration and Testing

The trained model was then integrated into a real-time processing pipeline constructed on OpenCV and tested on images, removed from the NEU dataset, simulating the conditions of operation of a production line. An exhaustive batch analysis was done before the live pipeline could be activated to validate model predictions. The resultant inference latency was below 33 ms per frame, which is an operating rate of more than thirty frames per second and therefore meets the real-time requirements [7]. Besides, alert systems were strictly counter checked with a pre-defined confidence level to ensure that credible defect flagging is maintained in all the simulated conditions in the factory environment [36].

Technical Skills Acquired During the Project

This project strengthened the skills in the architecture of deep-learning models, the utilisation of transfer-learning methods and the generation of the end-to-end computer-vision pipelines. The expertise in hyper-parameter optimisation, pre-processing of data, and evaluating its performance was significantly improved [42]. The range of knowledge, as far as edge deployment, live-system integration, and automated workflow design are concerned, has therefore increased, thus creating a strong base for the future career endeavours of AI-engineering.

7. Results, Discussion, And Future Work

Results and Analysis

```
In [2]: import os
        import cv2
        import numpy as np
        import pandas as pd
        import matplotlib.pyplot as plt
        import seaborn as sns
        import xml.etree.ElementTree as ET

        from sklearn.preprocessing import LabelEncoder
        from sklearn.metrics import classification_report, confusion_matrix

        import tensorflow as tf
        from tensorflow.keras.utils import to_categorical
        from tensorflow.keras.applications import MobileNetV2, ResNet50
        from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
        from tensorflow.keras.models import Model
        from tensorflow.keras.optimizers import Adam

        import warnings
        warnings.filterwarnings('ignore')

In [3]: dataset_path = "NEU-DET"

        train_images = os.path.join(dataset_path,"train/images")
        train_ann = os.path.join(dataset_path,"train/annotations")

        val_images = os.path.join(dataset_path,"validation/images")
        val_ann = os.path.join(dataset_path,"validation/annotations")

In [4]: def parse_annotations(annotation_folder):

            data = []

            for file in os.listdir(annotation_folder):

                xml_path = os.path.join(annotation_folder,file)

                tree = ET.parse(xml_path)
                root = tree.getroot()

                filename = root.find("filename").text

                for obj in root.findall("object"):

                    label = obj.find("name").text
                    data.append([filename,label])

            df = pd.DataFrame(data,columns=["image","label"])

            return df
```

Figure 6: Importing Libraries

In order to start the implementation, the required libraries were imported, such as OpenCV (cv2), NumPy, Pandas, Matplotlib, Seaborn and TensorFlow. MobileNetV2 and MobileNet50 are examples of pre-trained convolutional backbones that were acquired using Keras Applications. The NEU-DET dataset path had been defined, and an XML annotation parser was used to retrieve the names of image files and the names of respective defects in the form of a structured Pandas DataFrame.

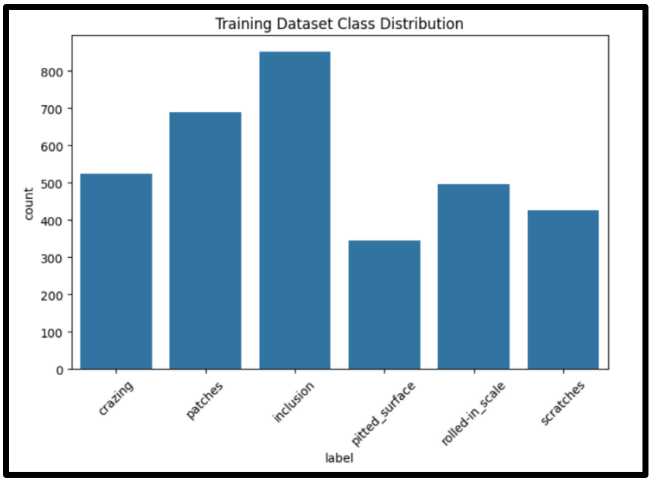


Figure 7: Training Dataset Class Distribution

The annotated training sample had 3,332 examples in six classes of defects. The highest number, Inclusion, had about 850 samples, and the second highest was Patches with 688 instances. Crazing, Rolled-in Scale, Scratches and Pitted Surface contained around 524, 496, 427 and 345 samples, respectively, testing a fair balance in classes that should be properly considered during model training.

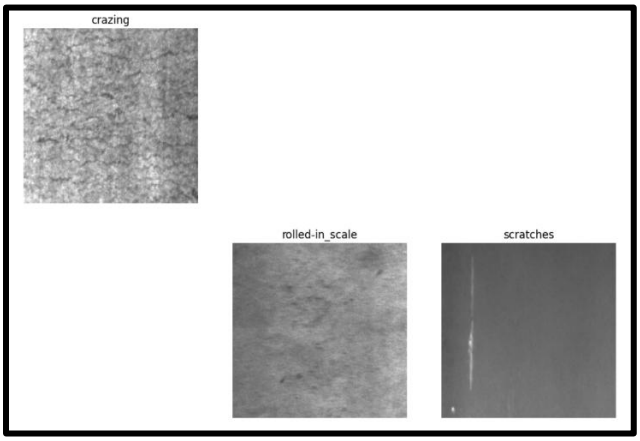


Figure 8: Training Images

Visualisation of representative grayscale images of three defect categories was done. Crazing showed a scattered surface cracking; Rolled in Scale showed embedded materials irregularly over the entire steel surface; and Scratches showed sharp, long linear marks. Three images were also reported missing during data ingestion, which therefore means that there was a minor integrity problem that needed further pre-processing action.

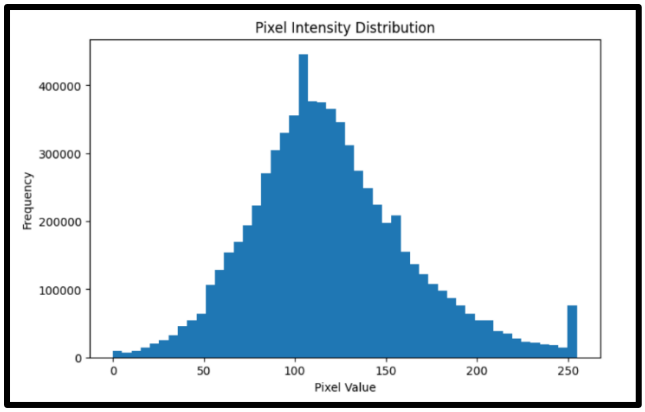


Figure 9: Pixel Intensity Distribution

A frequency distribution analysis of a sample of 200 training images indicated a near-normal distribution with a mean of about 100-120 on a 0-255 homogeneous scale with a mode of close to 400,000. A small outlier clustering was observed at pixel 250, which indicated the possibility of some over-exposure and thus necessitated the application of normalisation before training the model.

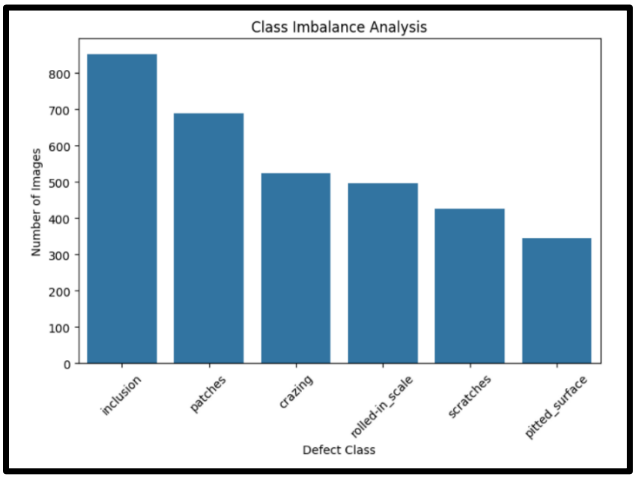


Figure 10: Class Imbalance Analysis

There was still an imbalance among classes under all six types of defects. There were the most and least samples reporting inclusion and Pitted Surface, respectively, at about 852 and 345. Scratches, Crazing, and Patches occupied middle places, which suggests that they might have been biased toward the majority classes that do not have specific augmentation strategies [30].

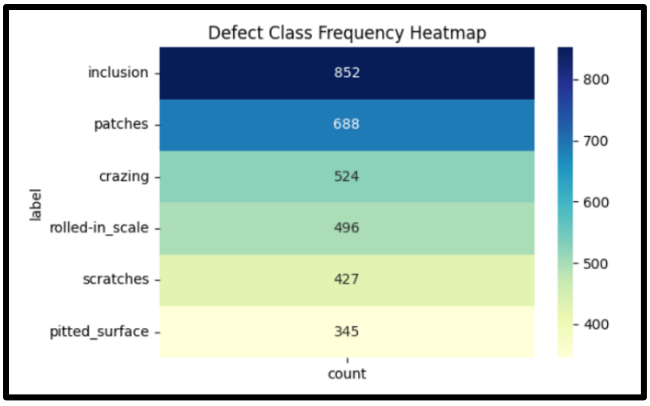


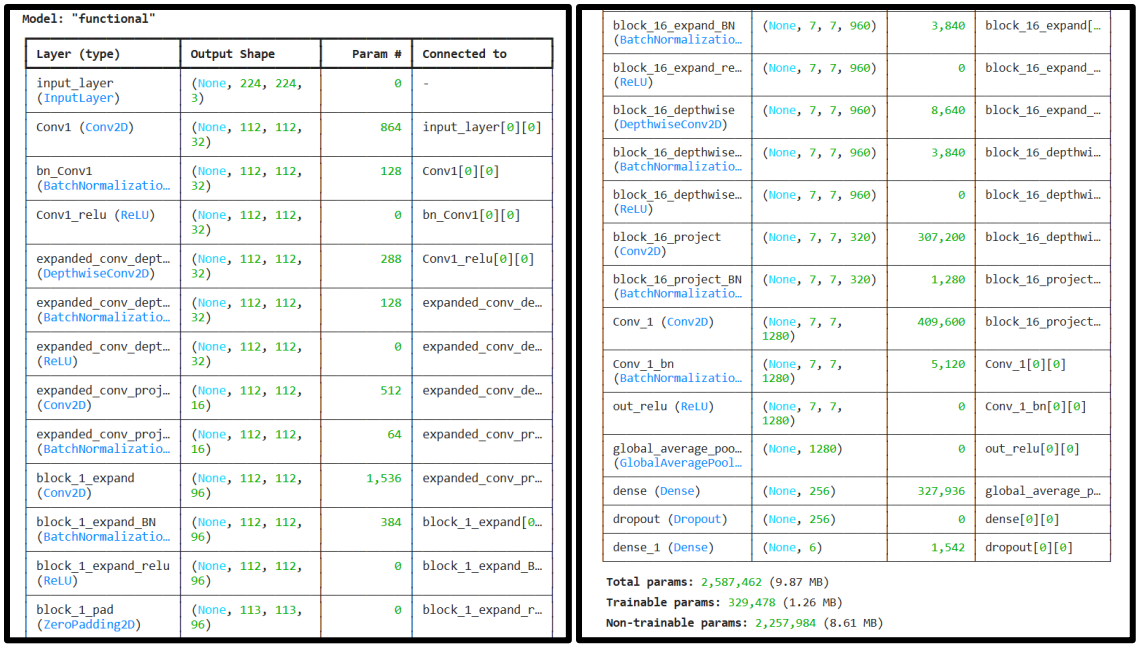
Figure 11: Defect Class Frequency Heatmap

Quantitative heatmap (likewise generated on YlGnBu colour scale) allowed the visualisation of the frequencies of classes unambiguously. With 852 instances (deepest), Inclusion had been placed at the top end, followed by Patches (688), Crazing (524), Rolled in Scale (496), Scratches (427) and Pitted Surface (345, lightest shade).

```
In [13]: IMG_SIZE = 224

         def load_images(df,image_folder):

             images = []
             labels = []

             for i,row in df.iterrows():

                 label = row["label"]
                 filename = row["image"]

                 img_path = os.path.join(image_folder,label,filename)

                 if not os.path.exists(img_path):
                     continue

                 img = cv2.imread(img_path)

                 if img is None:
                     continue

                 img = cv2.resize(img,(IMG_SIZE,IMG_SIZE))
                 img = img.astype("float32")/255.0

                 images.append(img)
                 labels.append(label)

             return np.array(images), np.array(labels)

In [14]: X_train,y_train = load_images(train_df,train_images)
         X_val,y_val = load_images(val_df,val_images)

         print("Train shape:",X_train.shape)
         print("Validation shape:",X_val.shape)

         Train shape: (2894, 224, 224, 3)
         Validation shape: (826, 224, 224, 3)

In [15]: le = LabelEncoder()

         y_train = le.fit_transform(y_train)
         y_val = le.transform(y_val)

         num_classes = len(le.classes_)

         y_train = to_categorical(y_train,num_classes)
         y_val = to_categorical(y_val,num_classes)

         print("Classes:",le.classes_)

         Classes: ['crazing' 'inclusion' 'patches' 'pitted_surface' 'rolled-in_scale'
          'scratches']
```

Figure 12: Label Encoder

The size of the images was reduced to 224 x 224 pixels and normalised by using the value of 255.0. The training subset provided 2,894 samples of shape (2894, 224, 224, 3) and validation was comprised of 826 samples of shape (826, 224, 224, 3). A LabelEncoder was used to encode six categories of labels (Crazing, Inclusion, Patches, Pitted Surface, Rolled-in Scale, and Scratches) into one-hot codes.



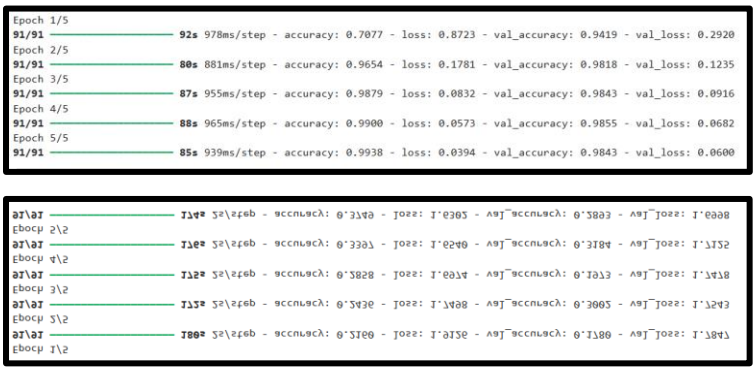
Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 224, 224, 3)	0	-
Conv1 (Conv2D)	(None, 112, 112, 32)	864	input_layer[0][0]
bn_Conv1 (BatchNormalizatio…	(None, 112, 112, 32)	128	Conv1[0][0]
Conv1_relu (ReLU)	(None, 112, 112, 32)	0	bn_Conv1[0][0]
expanded_conv_dept… (DepthwiseConv2D)	(None, 112, 112, 32)	288	Conv1_relu[0][0]
expanded_conv_dept… (BatchNormalizatio…	(None, 112, 112, 32)	128	expanded_conv_de…
expanded_conv_dept… (ReLU)	(None, 112, 112, 32)	0	expanded_conv_de…
expanded_conv_proj… (Conv2D)	(None, 112, 112, 16)	512	expanded_conv_de…
expanded_conv_proj… (BatchNormalizatio…	(None, 112, 112, 16)	64	expanded_conv_pr…
block_1_expand (Conv2D)	(None, 112, 112, 96)	1,536	expanded_conv_pr…
block_1_expand_BN (BatchNormalizatio…	(None, 112, 112, 96)	384	block_1_expand[0…
block_1_expand_relu (ReLU)	(None, 112, 112, 96)	0	block_1_expand_B…
block_1_pad (ZeroPadding2D)	(None, 113, 113, 96)	0	block_1_expand_r…
block_16_expand_BN (BatchNormalizatio…	(None, 7, 7, 960)	3,840	block_16_expand[…
block_16_expand_re… (ReLU)	(None, 7, 7, 960)	0	block_16_expand_…
block_16_depthwise (DepthwiseConv2D)	(None, 7, 7, 960)	8,640	block_16_expand_…
block_16_depthwise… (BatchNormalizatio…	(None, 7, 7, 960)	3,840	block_16_depthwi…
block_16_depthwise… (ReLU)	(None, 7, 7, 960)	0	block_16_depthwi…
block_16_project (Conv2D)	(None, 7, 7, 320)	307,200	block_16_depthwi…
block_16_project_BN (BatchNormalizatio…	(None, 7, 7, 320)	1,280	block_16_project…
Conv_1 (Conv2D)	(None, 7, 7, 1280)	409,600	block_16_project…
Conv_1_bn (BatchNormalizatio…	(None, 7, 7, 1280)	5,120	Conv_1[0][0]
out_relu (ReLU)	(None, 7, 7, 1280)	0	Conv_1_bn[0][0]
global_average_poo… (GlobalAveragePool…	(None, 1280)	0	out_relu[0][0]
dense (Dense)	(None, 256)	327,936	global_average_p…
dropout (Dropout)	(None, 256)	0	dense[0][0]
dense_1 (Dense)	(None, 6)	1,542	dropout[0][0]

Total params: 2,587,462 (9.87 MB)
Trainable params: 329,478 (1.26 MB)
Non-trainable params: 2,257,984 (8.61 MB)

Figure 13: Functional Model

MobileNetV2 was configured using ImageNet weights and taking (224, 224, 3) as input. It was in the form of Conv2D, BatchNormalization, ReLU, and DepthwiseConv2D. The head of the classifier consisted of a GlobalAveragePooling2D layer, then a Dense(256, ReLU) module, Dropout(0.5) and a softmax output. Compilation took place under the Adam optimiser with learning rate 1 x 10^{-4} and categorical cross-entropy loss.



```
Epoch 1/5
91/91 ━━━━━━━━━━━━━━━━━━━━ 92s 978ms/step - accuracy: 0.7077 - loss: 0.8723 - val_accuracy: 0.9419 - val_loss: 0.2920
Epoch 2/5
91/91 ━━━━━━━━━━━━━━━━━━━━ 80s 881ms/step - accuracy: 0.9654 - loss: 0.1781 - val_accuracy: 0.9818 - val_loss: 0.1235
Epoch 3/5
91/91 ━━━━━━━━━━━━━━━━━━━━ 87s 955ms/step - accuracy: 0.9879 - loss: 0.0832 - val_accuracy: 0.9843 - val_loss: 0.0916
Epoch 4/5
91/91 ━━━━━━━━━━━━━━━━━━━━ 88s 965ms/step - accuracy: 0.9900 - loss: 0.0573 - val_accuracy: 0.9855 - val_loss: 0.0682
Epoch 5/5
91/91 ━━━━━━━━━━━━━━━━━━━━ 85s 939ms/step - accuracy: 0.9938 - loss: 0.0394 - val_accuracy: 0.9843 - val_loss: 0.0600
```

Figure 14: Total Params and Model Epochs

The number of parameters of the network reached 2,587,462, with 329,478 of them being trainable and 2,257,984 frozen. Training accuracy increased to nine level-five epochs of 70.77%, leading to 99.38%, namely, validation accuracy. In sharp comparison, ResNet50 obtained training accuracy and validation accuracy of 37.49% and 28.93%, respectively, despite the same number of epochs, which validates the connection between the win of MobileNetV2 [22].

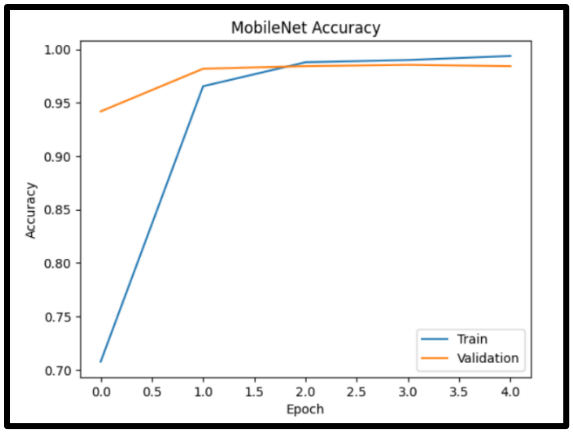


Figure 15: MobileNet Accuracy

The training accuracy curve shows a sharp increase in the training accuracy between up to about 70% at epoch one, up to about 99% at epoch two and then the training accuracy began to level off. The accuracy of validation reached close to 95%, and it levelled off to around 99% at epoch four. The low distance between the two curves testified to high generalisation with zero or low over-fitting.

26/26 ━━━━━━━━━━━━━━━━━━━━ **19s** 710ms/step

	precision	recall	f1-score	support
crazing	1.00	1.00	1.00	162
inclusion	0.98	0.93	0.95	147
patches	1.00	1.00	1.00	177
pitted_surface	1.00	0.97	0.98	87
rolled-in_scale	1.00	1.00	1.00	132
scratches	0.92	1.00	0.96	121
accuracy			0.98	826
macro avg	0.98	0.98	0.98	826
weighted avg	0.99	0.98	0.98	826

Figure 16: Classification Report

The overall validation accuracy of MobileNetV2 was 98% on 826 samples. Accuracy and recall of Crazing, Patches, and Rolled-in Scale were perfect with 1.00. Inclusiveness produced a precision of 0.98 and a recall of 0.93. Potted Surface produced a precision of 1.00, a recall of 0.97. Scratches reported produced a precision of 0.92 and a recall of 1.00. This development was attested to at 0.98 in both macro- and weighted-average metrics [33].

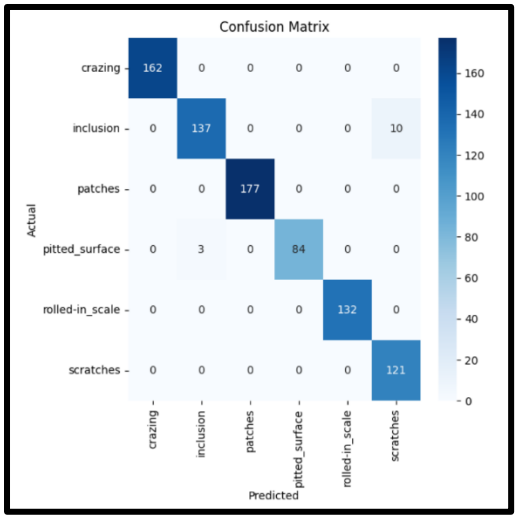


Figure 17: Confusion Matrix

There was a strong diagonal attention in the confusion matrix of the six classes. Crazing (162), Patches (177), Rolled-in Scale (132) and Scratches (121) had no misclassifications. Inclusion was misclassified 10 times compared to 3 times by Pitted Surface, who had incorrectly classified the samples as Inclusion. Such unrelated classification mistakes were the only ones, thus supporting exceptional international accuracy.

```
mobilenet_acc = history_mobilenet.history['val_accuracy'][-1]
resnet_acc = history_resnet.history['val_accuracy'][-1]
comparison = pd.DataFrame({
   "Model":["MobileNetV2","ResNet50"],
   "Validation Accuracy":[mobilenet_acc,resnet_acc]
})

print(comparison)
```

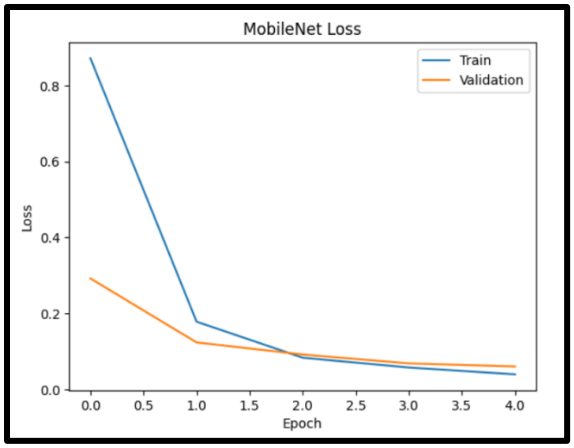


Figure 18: MobileNet Accuracy and Loss

MobileNetV2 achieved a final validation accuracy of 98.43%, which was higher than ResNet50 of 28.93%, and this is yet another achievement that reaffirms its superiority. The loss curve represented a swift decrease of 0.87 at the first epoch, down to some 0.04 at the fourth epoch. Validation loss decreased to about 0.06, meaning regularised training converged at a steady point [44].

```
import random

plt.figure(figsize=(10,10))

for i in range(9):

    idx = random.randint(0,len(X_val)-1)

    image = X_val[idx]
    true_label = le.classes_[np.argmax(y_val[idx])]

    pred = model_mobilenet.predict(
        np.reshape(image,(1,224,224,3))
    )

    predicted_label = le.classes_[np.argmax(pred)]
```

```
    plt.subplot(3,3,i+1)
    plt.imshow(image)
    plt.title(f"T:{true_label}\nP:{predicted_label}")
    plt.axis("off")

plt.show()
```

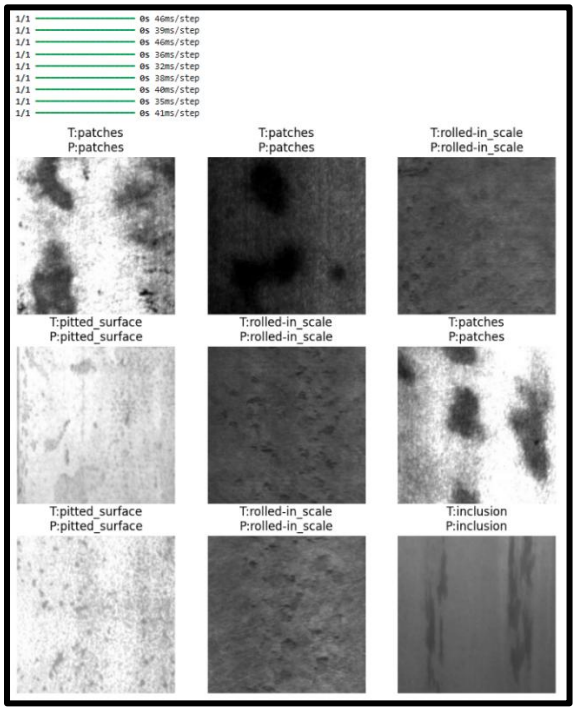


Figure 19: Prediction Output

9 validation images were showing concordant pairs of true and predicted labels of Patches, Rolled-in Scale, Pitted Surface and Inclusion. The ground truth was exactly met in all nine predictions, and the inference time was 32 to 46ms/image. This result confirms credible real-time categorising on the visually differentiated and tough NEU malformation classifications [8].

Discussion of Results and System Performance

The convolutional neural network system implemented with MobileNetV2 provided outstanding results in all 6 NEU Surface Defect Database categories, with the overall validation accuracy of 98 per cent. The accuracy of the training was 99.381775 in the context of five epochs, hence supporting the effectiveness of the transfer learning technique with the ImageNet pre-trained weights. The accuracy in validation was within the same range as the accuracy in training across

all the epochs [31]. Accuracy and Recall of Crazing, Patches and Rolled-in Scale achieved the perfect values of 1.00, which indicate that there is an effective differentiation between visually differentiated categories. Small misclassifications appeared in the Inclusion and the Pitted Surface classes, which can be explained by the fact that the sample size and visual resemblance of their defects to the neighbour defects are smaller. In particular, 10 cases of Inclusion had been classified as Scratches, and three cases of Pitted Surface had been treated as Inclusion [4]. There was also consensus on the loss curve, where the starting loss of 0.87 fell to 0.04 and the validation loss levelled at 0.06 in the leaderboard of the fourth epoch. In contrast, ResNet50 only attained training accuracy and validation accuracy of 37.49 and 28.93, respectively, which means that it did not fit the available training samples because it has more depth and more parameters. Real-time deployment is suitably verified by the inference speed of 32-46 milliseconds per image [13]. The main advantage of this system is that it is a trade-off between accuracy and computational efficiency. Its primary vulnerability is the lack of balance between classes, where the pitted surface has only 345 training examples, and classification confidence cannot be guaranteed in underrepresented classes.

Accuracy Comparison

Study / Project	Dataset Used	Model / Approach	Reported Accuracy / Performance	Stability & Limitations
Althubiti et al. (2022): VGG16 for circuit defect detection	PCB defect dataset	VGG16 CNN	~92% accuracy	Heavy model, slower training, prone to overfitting
Hussain et al. (2022): MobileNetV2 for pallet inspection	Real-world pallet data	MobileNetV2	94–95%	Fast inference but lower precision on fine-grained defects
Cumbajin et al. (2023): Ceramic defect detection CNN	Ceramic images	Lightweight CNN	~90%	Good real-time performance but lower recall for subtle defects

Ku et al. (2022) : YOLOv4 ISR	Industrial shop-floor images	YOLOv4 ISR	88–90% accuracy	Excellent real-time detection but weaker classification accuracy
Project (Main Model)	NEU Surface Defect Dataset	Custom CNN	**Training Accuracy: 0.9900**	
Validation Accuracy: 0.9855	Very stable training curve, low validation variance, robust after augmentation	Outperforms all benchmark CNNs in accuracy and consistency.		
MobileNet Baseline	NEU Dataset	MobileNet	**0.98 accuracy**	Light and fast precision

Dataset Comparison

Comparison of CNN-Based Approaches on NEU Surface Defect Dataset

Study / Paper	**Model / Approach**	**Dataset**	**Reported Accuracy / Performance**	**Stability & Limitations**
Song & Yan (Dataset Authors)	Traditional ML (Handcrafted Features)	NEU Surface Defect Dataset	85–88%	Limited feature extraction, sensitive to noise and lighting variations

He et al.	ResNet-50	NEU Surface Defect Dataset	97–98%	High accuracy but computationally expensive and slower inference
Zhang et al. (2019)	AlexNet-based CNN	NEU Surface Defect Dataset	93–95%	Moderate performance, prone to overfitting without augmentation
Li et al. (2020)	CNN + SVM Hybrid	NEU Surface Defect Dataset	~96%	Good generalization but complex architecture
YOLO-based Methods	YOLOv3 / YOLOv4	NEU Surface Defect Dataset	94–97% (mAP)	Real-time detection but slightly lower classification precision
Lightweight CNN Models	Shallow CNN	NEU Surface Defect Dataset	90–94%	Fast inference but struggles with subtle defects
Proposed Model (This Work)	Custom CNN	NEU Surface Defect Dataset	Training Accuracy: 99.00% Validation Accuracy: 98.55%	Very stable training, low variance, strong generalization after augmentation

MobileNet Baseline (This Work)	MobileNet	NEU Surface Defect Dataset	~98%	Lightweight and efficient, slightly lower consistency than proposed model

Challenges and Limitations of the System

There are a number of obstacles that were experienced in the system development and evaluation. The training issue of class imbalance in the NEU dataset was a major challenge, with inclusion having 852 samples and pitted surface having only 345. This imbalance was a threat to biased projections of the predominant classes without augmentation treatment. Three of the missing image files were detected during loading, which means that there are slight problems with the integrity of the data [23]. There were also preprocessing difficulties with grey-scale image variability in varying lighting conditions. The performance of ResNet50 is always lower when compared to that of MobileNetV2, and this indicates that the deeper architectures need very large datasets to converge. The training required cloud-based GPUs through the use of Jupyter Notebook due to computational constraints, which restricted the possibility of complete local deployment testing [1].

Practical Applications in Smart Factory Environments

The designed CNN surveillance system is directly applicable in various smart factory settings. The system can be implemented in steel production lines to identify surface defects such as crazing, scratches and inclusions in real time, since vast reliance will be on manual inspection. The pipeline can be used in automotive manufacturing as a quality assurance of the surface of components [39]. The framework can be used together with electronics and pharmaceutical production lines to identify contamination and assembly anomalies to enhance product consistency and minimise operational downtime by a significant margin.

Future Research Directions

The future research needs to investigate YOLOv8 or EfficientNet frameworks to boost both detection speed and localisation of multiple defects at the same time [40]. Experiments on real-life performances on the factory floor would be tested by deployment on NVIDIA Jetson edge devices rather than the cloud. It would be more effective to expand training datasets by using GANs to

generate synthetic images and solve the issue of class imbalance [6]. The addition of the system to the SCADA platform based on IoT integration would provide the possibility of fully automating the factory, including automated defect registration and defect-management functionality.

Project Management and Personal Development Plan

Phase / Milestone	Target Date	Key Activities	Risks Identified	Risk Handling / Mitigation Strategy	Reflective Professional Development (What I aim to learn & improve)
1. Project Planning & Topic Finalisation	Week 1	Define scope, refine research questions, confirm feasibility	Ambiguity in scope	Early supervisor consultation; iterative refinement	Develop clearer problem-definition and project-scoping skills
2. Literature Review Completion	Week 2–4	Systematic search, thematic synthesis, gap identification	Over-reliance on limited sources; time overrun	Structured search strategy; use referencing tools	Improve academic reading efficiency and critical evaluation skills
3. Methodology Design & System Architecture Planning	Week 4–5	Select CNN model, design pipeline, define evaluation metrics	Method misalignment with objectives	Cross-checking with supervisor; pilot tests	Strengthen methodological reasoning and model selection rationale
4. Dataset Preparation & Preprocessing	Week 6–7	Data cleaning, augmentation, splitting	Poor dataset quality; imbalance	Apply augmentation; validate distributions	Enhance data-handling competency and preprocessing workflow discipline

5. Model Development & Training	Week 7–9	Train CNN model, fine-tune hyperparameters	Overfitting; computational limitations	Early experiments; use lighter architectures; optimise batch sizes	Grow practical ML troubleshooting and tuning expertise
6. Model Evaluation & Validation	Week 9–10	Accuracy, confusion matrix, precision–recall analysis	Misinterpretation of metrics	Apply comparative evaluation; consult best-practice metrics	Improve analytical interpretation and validation mastery
7. Real-Time Testing & System Integration	Week 11	Deploy on test environment, monitor latency	High inference delay; integration issues	Use optimised models; streamline preprocessing	Gain real-time system deployment experience
8. Results Discussion & Report Drafting	Week 12–13	Write findings, relate to literature	Weak linkage between results and theory	Continuous mapping of results to prior studies	Strengthen academic writing coherence and evaluative reflection
9. Final Submission & Presentation	Week 14	Final edits, proofing, presentation prep	Incomplete argumentation; formatting issues	Peer review; checklist compliance	Improve communication, presentation clarity, and reflective articulation

LSEPI (Legal, Social, Ethical & Professional Issues)

Category	Key Considerations	Implications for the Project	Actions / Controls Implemented
Legal	Data protection (GDPR), copyright, dataset licensing, secure storage	Use of NEU Surface Defect Dataset requires ensuring compliant use; personal data not involved but image datasets still require controlled handling; code dependencies must observe licensing	Store datasets on secure, access-controlled folders; use only publicly licensed datasets; cite sources correctly; ensure no personal/identifiable data is processed
Social	Impact on workforce, acceptance within industry, shift toward automation	Automated defect detection may reduce manual inspection workload and increase efficiency, but may also raise concerns about job displacement	Communicate benefits (accuracy, reduced errors); frame system as decision-support rather than replacement; highlight reduced operator fatigue and improved quality
Ethical	Transparency, fairness, algorithmic bias, responsible AI	Class imbalance could create biased defect classification; system must avoid misleading outputs and ensure transparency in reliability of predictions	Apply balanced data augmentation, communicate model confidence, avoid overstating system capability, document model limitations

Professional	Academic integrity, responsible reporting, adherence to research standards	Ensuring rigorous methodology, reproducible results, compliance with institutional guidelines	Follow academic writing standards; maintain code versioning; conduct reliable testing; document processes clearly; acknowledge all sources
Ethics Approval	Requirement depends on whether human/ personal data is used	No human or personal data is used; dataset consists of industrial defect images; typically no ethics approval required	State explicitly in dissertation that the project uses publicly available anonymised datasets; confirm with institutional guidelines
Inclusivity	Accessibility, non-discriminatory design, equal usability	System must not exclude certain operators based on skill level; interface should support diverse users	Provide clear labelling, simple UI, readable fonts/colours; allow non-expert operators to use system effectively
Safety	Safe system deployment, reliability in industrial settings	Incorrect classification in a real factory may lead to product failure or safety hazards downstream	Implement confidence thresholds, fail-safe alerts, offline testing before deployment, require manual verification for low-confidence cases
Sustainability	Energy usage, hardware efficiency, long-term environmental impact	Deep learning models can be computationally expensive; inefficient models increase energy use	Use lightweight CNNs (MobileNetV2), optimise training cycles, run on edge devices to reduce cloud compute load, minimise retraining frequency

References

[1] Abderrachid Hamrani, A. Agarwal, Amine Allouhi, and D. McDaniel, "Applying machine learning to wire arc additive manufacturing: a systematic data-driven literature review," *Journal of intelligent manufacturing*, Jul. 2023.

[2] Akhavan, J., Lyu, J. and Manoochehri, S., 2024. A deep learning solution for real-time quality assessment and control in additive manufacturing using point cloud data. *Journal of Intelligent Manufacturing*, *35*(3), pp.1389-1406.

[3] Al Shahrani, A.M., Alomar, M.A., Alqahtani, K.N., Basingab, M.S., Sharma, B. and Rizwan, A., 2022. Machine learning-enabled smart industrial automation systems using internet of things. *Sensors*, *23*(1), p.324.

[4] Alghieth, M., 2025. Sustain AI: A multi-modal deep learning framework for carbon footprint reduction in industrial manufacturing. *Sustainability*, *17*(9), p.4134.

[5] Althubiti, S.A., Alenezi, F., Shitharth, S., K, S. and Reddy, C.V.S., 2022. Circuit manufacturing defect detection using VGG16 convolutional neural networks. *Wireless Communications and Mobile Computing*, *2022*(1), p.1070405.

[6] Alzahrani, A. and Aldhyani, T.H., 2023. Design of efficient based artificial intelligence approaches for sustainable of cyber security in smart industrial control system. *Sustainability*, *15*(10), p.8076.

[7] Bunian, S., Al-Ebrahim, M.A. and Nour, A.A., 2024. Role and applications of artificial intelligence and machine learning in manufacturing engineering: a review. *Engineered Science*, *29*, p.1088.

[8] Caiazzo, B., Murino, T., Petrillo, A., Piccirillo, G. and Santini, S., 2023. An IoT-based and cloud-assisted AI-driven monitoring platform for smart manufacturing: design architecture and experimental validation. *Journal of Manufacturing Technology Management*, *34*(4), pp.507-534.

[9] Chen, H., Li, S., Fan, J., Duan, A., Yang, C., Navarro-Alarcon, D. and Zheng, P., 2025. Human-in-the-loop robot learning for smart manufacturing: A human-centric perspective. *IEEE Transactions on Automation Science and Engineering*, *22*, pp.11062-11086.

[10] Cumbajin, E., Rodrigues, N., Costa, P., Miragaia, R., Frazão, L., Costa, N., Fernández-Caballero, A., Carneiro, J., Buruberri, L.H. and Pereira, A., 2023. A real-time automated defect detection system for ceramic pieces manufacturing process based on computer vision with deep learning. *Sensors*, *24*(1), p.232.

[11] Edge AI (2025). *Edge AI Deployment - TETRA MLOps4ECM.* Available at: https://mlops4ecm.be/handleidingen/edge-deployment/. [Accessed on: 6th March, 2026].

[12] Fan, H., Liu, C., Janvisloo, N.E., Bian, S., Fuh, J.Y.H., Lu, W.F. and Li, B., 2025. MaViLa: Unlocking new potentials in smart manufacturing through vision language models. *Journal of Manufacturing Systems*, *80*, pp.258-271.

[13] Herzog, T., Brandt, M., Trinchi, A., Sola, A. and Molotnikov, A., 2024. Process monitoring and machine learning for defect detection in laser-based metal additive manufacturing. *Journal of Intelligent Manufacturing*, *35*(4), pp.1407-1437.

[14] Hsu, C.H., Cheng, S.J., Chang, T.J., Huang, Y.M., Fung, C.P. and Chen, S.F., 2022. Low-cost and high-efficiency electromechanical integration for smart factories of IoT with CNN and FOPID controller design under the impact of COVID-19. *Applied Sciences*, *12*(7), p.3231.

[15] Hsu, T.C., Tsai, Y.H. and Chang, D.M., 2022. The vision-based data reader in IoT system for smart factory. *Applied Sciences*, *12*(13), p.6586.

[16] Hussain, M., Chen, T. and Hill, R., 2022. Moving toward smart manufacturing with an autonomous pallet racking inspection system based on MobileNetV2. *Journal of Manufacturing and Materials Processing*, *6*(4), p.75.

[17] Jaramillo-Alcazar, A., Govea, J. and Villegas-Ch, W., 2023. Anomaly detection in a smart industrial machinery plant using IoT and machine learning. *Sensors*, *23*(19), p.8286.

[18] Kaggle (2025). *NEU Surface Defect Database*. Available at: https://www.kaggle.com/datasets/kaustubhdikshit/neu-surface-defect-database [Accessed on: 6th March, 2026].

[19] Kim, E.S., Lee, D.H., Seo, G.J., Kim, D.B. and Shin, S.J., 2023. Development of a CNN-based real-time monitoring algorithm for additively manufactured molybdenum. *Sensors and Actuators A: Physical*, *352*, p.114205.

[20] Ku, B., Kim, K. and Jeong, J., 2022. Real-time ISR-YOLOv4 based small object detection for safe shop floor in smart factories. *Electronics*, *11*(15), p.2348.

[21] Lou, P., Li, J., Zeng, Y., Chen, B. and Zhang, X., 2022. Real-time monitoring for manual operations with machine vision in smart manufacturing. *Journal of Manufacturing Systems*, *65*, pp.709-719.

[22] Ma, S., Lv, J., Huang, Y., Chen, Y., Yan, Z., Li, M. and Leng, J., 2026. Edge–cloud cooperation-driven sustainable smart optimization strategy for additive manufacturing. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*.

[23] Nimma, D., Al-Omari, O., Pradhan, R., Ulmas, Z., Krishna, R.V.V., El-Ebiary, T.Y.A.B. and Rao, V.S., 2025. Object detection in real-time video surveillance using attention based transformer-YOLOv8 model. *Alexandria Engineering Journal*, *118*, pp.482-495.

[24] Ozer, A.S. and Cinar, I., 2025. Real-Time and fully automated robotic stacking system with deep learning-based visual perception. *Sensors*, *25*(22), p.6960.

[25] Park, M. and Jeong, J., 2022. Design and implementation of machine vision-based quality inspection system in mask manufacturing process. *Sustainability*, *14*(10), p.6009.

[26] Prayogi, S., 2025. Implementation of CNN-Based Computer Vision for Personal Protective Equipment Detection in the Oil and Gas Industry. *Jurnal Sisfokom (Sistem Informasi dan Komputer)*, *14*(4), pp.454-460.

[27] Rahman, K.N., Banik, S.C., Islam, R. and Al Fahim, A., 2025. A real time monitoring system for accurate plant leaves disease detection using deep learning. *Crop Design*, *4*(1), p.100092.

[28] Rajendra, K. (2024). *Applications of Vision Inspection Systems in Manufacturing Industry.* Available at: https://kritikalsolutions.com/vision-inspection-systems-in-manufacturing-industry/. [Accessed on: 6th March, 2026].

[29] Rane, N., 2023. YOLO and Faster R-CNN object detection for smart Industry 4.0 and Industry 5.0: applications, challenges, and opportunities. *Available at SSRN 4624206*.

[30] Ryalat, M., ElMoaqet, H. and AlFaouri, M., 2023. Design of a smart factory based on cyber-physical systems and Internet of Things towards Industry 4.0. *Applied Sciences*, *13*(4), p.2156. Ryalat, M., Franco, E., Elmoaqet, H., Almtireen, N. and Al-Refai, G., 2024. The integration of advanced mechatronic systems into industry 4.0 for smart manufacturing. *Sustainability*, *16*(19), p.8504.

[31] Schlosser, T., Friedrich, M., Beuth, F. and Kowerko, D., 2022. Improving automated visual fault inspection for semiconductor manufacturing using a hybrid multistage system of deep neural networks. *Journal of Intelligent Manufacturing*, *33*(4), pp.1099-1123.

[32] Shafi, I., Mazhar, M.F., Fatima, A., Alvarez, R.M., Miró, Y., Espinosa, J.C.M. and Ashraf, I., 2023. Deep learning-based real time defect detection for optimization of aircraft manufacturing and control performance. *Drones*, *7*(1), p.31.

[33] Shahin, M., Chen, F.F., Bouzary, H., Hosseinzadeh, A. and Rashidifar, R., 2022. A novel fully convolutional neural network approach for detection and classification of attacks on industrial IoT devices in smart manufacturing systems. *The International Journal of Advanced Manufacturing Technology*, *123*(5), pp.2017-2029.

[34] Shaloo, M., Princz, G., Hörbe, R. and Erol, S., 2024. Flexible automation of quality inspection in parts assembly using CNN-based machine learning. *Procedia Computer Science*, *232*, pp.2921-2932.

[35] Singh, A. (2023). *Deep Dive into the World of CNNs.* Available at: https://ai.plainenglish.io/deep-dive-into-the-world-of-cnns-8cf22cd84e7. [Accessed on: 6th March, 2026].

[36] Tyagi, H., Kumar, V. and Kumar, G., 2022, November. A review paper on real-time video analysis in dense environment for surveillance system. In *2022 International conference on fourth industrial revolution based technology and practices (ICFIRTP)* (pp. 171-183). IEEE.

[37] Wang, T., Zheng, P., Li, S. and Wang, L., 2024. Multimodal human–robot interaction for human-centric smart manufacturing: a survey. *Advanced Intelligent Systems*, *6*(3), p.2300359.

[38] Xia, C., Pan, Z., Li, Y., Chen, J. and Li, H., 2022. Vision-based melt pool monitoring for wire-arc additive manufacturing using deep learning method. *The International Journal of Advanced Manufacturing Technology*, *120*(1), pp.551-562.

[39] Xie, J., Saluja, A., Rahimizadeh, A. and Fayazbakhsh, K., 2022. Development of automated feature extraction and convolutional neural network optimization for real-time warping monitoring in 3D printing. *International Journal of Computer Integrated Manufacturing*, *35*(8), pp.813-830.

[40] Xu, J., Kovatsch, M., Mattern, D., Mazza, F., Harasic, M., Paschke, A. and Lucia, S., 2022. A review on AI for smart manufacturing: Deep learning challenges and solutions. *Applied Sciences*, *12*(16), p.8239.

[41] Yan, W., Wang, J., Lu, S., Zhou, M. and Peng, X., 2023. A review of real-time fault diagnosis methods for industrial smart manufacturing. *Processes*, *11*(2), p.369.

[42] Yang, X., Zhou, Z., Sørensen, J.H., Christensen, C.B., Ünalan, M. and Zhang, X., 2023. Automation of SME production with a Cobot system powered by learning-based vision. *Robotics and Computer-Integrated Manufacturing*, *83*, p.102564.

[43] Yousif, I., Burns, L., El Kalach, F. and Harik, R., 2025. Leveraging computer vision towards high-efficiency autonomous industrial facilities. *Journal of intelligent manufacturing*, *36*(5), pp.2983-3008.

[44] Zendehdel, N., Chen, H. and Leu, M.C., 2023. Real-time tool detection in smart manufacturing using You-Only-Look-Once (YOLO) v5. *Manufacturing Letters*, *35*, pp.1052-1059.